

SIMATS ENGINEERING

ASSESSMENT TOOLS 2

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 25

Duration : 1 Hour

Experiments

Code of Conduct:

I Deepak V-192411021 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Deepak V

Question 1

Write a C program to check whether a given string belongs to the language defined by the following Context Free Grammar (CFG):

$S \rightarrow 0A1$ $A \rightarrow 0A \mid 1A \mid \epsilon$

Language generated: $0(0+1)^*1$

Pseudocode

```

BEGIN
  READ string S
  n ← LENGTH(S)
  IF n < 2 THEN
    OUTPUT "Not Accepted"; STOP
  IF S[0] != '0' OR S[n-1] != '1' THEN
    OUTPUT "Not Accepted"; STOP
  FOR i ← 1 TO n-2 DO
    IF S[i] != '0' AND S[i] != '1' THEN
      OUTPUT "Not Accepted"; STOP
  OUTPUT "Accepted"
END

```

Sample Output

main.c	Run	Output
<pre> 4 int main() { 20 if (str[0] != '0' str[n - 1] != '1') { 21 printf("String is NOT accepted.\n"); 22 return 0; 23 } 24 25 // Middle symbols can only be 0 or 1 26 for (i = 1; i < n - 1; i++) { 27 if (str[i] != '0' && str[i] != '1') { 28 printf("String is NOT accepted.\n"); 29 return 0; 30 } 31 } 32 33 printf("String is accepted.\n"); 34 35 return 0; 36 } </pre>		<pre> Enter a string: 01101 String is accepted. === Code Execution Successful === </pre>

Explanation

The grammar $S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \epsilon$ generates strings that begin with 0, end with 1, and allow any combination of 0s and 1s in between — the regular language $0(0+1)^*1$. The program checks only the first and last characters and treats everything in between as free. For the input 01101, the string starts with 0 and ends with 1, so it is Accepted.

Question 2

Write a C program to check whether a given string belongs to the language defined by the following Context Free Grammar (CFG):

$$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$$

Language generated: *Palindromes over {0,1}*

Pseudocode

```

BEGIN
  READ string S
  n ← LENGTH(S)
  FOR i ← 0 TO n-1 DO
    IF S[i] != '0' AND S[i] != '1' THEN
      OUTPUT "Not Accepted"; STOP
  i ← 0 ; j ← n-1
  WHILE i < j DO
    IF S[i] != S[j] THEN
      OUTPUT "Not Accepted"; STOP
    i ← i+1 ; j ← j-1
  OUTPUT "Accepted"
END

```

Sample Output

main.c	Run	Output
<pre> 4 int main() { 27 } 28 29 // Check all characters are 0 or 1 30 for (i = 0; i < strlen(str); i++) { 31 if (str[i] != '0' && str[i] != '1') { 32 palindrome = 0; 33 break; 34 } 35 } 36 37 if (palindrome) 38 printf("String is accepted.\n"); 39 else 40 printf("String is NOT accepted.\n"); 41 42 return 0; 43 } </pre>		<pre> Enter a string: 1001 String is accepted. --- Code Execution Successful --- </pre>

Explanation

The grammar $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$ generates all palindromes made of 0s and 1s, since each production mirrors a symbol on both ends of the string. The program compares characters from both ends moving inward. For the input 1001, the string reads the same forwards and backwards, so it is Accepted.

Question 3

Write a C program to check whether a given string belongs to the language defined by the following Context Free Grammar (CFG):

$$S \rightarrow 0S0 \mid A \quad A \rightarrow 1A \mid \epsilon$$

Language generated: $0^n 1^m 0^n$ ($n, m \geq 0$)

Pseudocode

```

BEGIN
  READ string S ; n <- LENGTH(S) ; i <- 0
  leftZeros <- 0 ; rightZeros <- 0
  WHILE i < n AND S[i] = '0' DO
    leftZeros <- leftZeros + 1 ; i <- i + 1
  WHILE i < n AND S[i] = '1' DO
    i <- i + 1
  WHILE i < n AND S[i] = '0' DO
    rightZeros <- rightZeros + 1 ; i <- i + 1
  IF i != n THEN
    OUTPUT "Not Accepted"; STOP
  IF leftZeros = rightZeros THEN
    OUTPUT "Accepted"
  ELSE
    OUTPUT "Not Accepted"
END

```

Sample Output

main.c	Run	Output
<pre> 4 int main() { 31 while (i < n && str[i] == '0') { 34 } 35 36 // If any other characters exist, reject 37 if (i != n) { 38 printf("String is NOT accepted.\n"); 39 return 0; 40 } 41 42 // Number of beginning and ending zeros must be equal 43 if (leftZeros == rightZeros) 44 printf("String is accepted.\n"); 45 else 46 printf("String is NOT accepted.\n"); 47 48 return 0; 49 } </pre>		<pre> Enter a string: 0011100 String is accepted. --- Code Execution Successful --- </pre>

Explanation

The grammar $S \rightarrow 0S0 \mid A$, $A \rightarrow 1A \mid \epsilon$ generates strings with equal blocks of 0s at the start and end, and any number of 1s in the middle — the language $0^n 1^m 0^n$. The program counts leading zeros, skips the middle 1s, then counts trailing zeros and compares the two counts. For the input 0011100, there are two leading zeros and two trailing zeros, so it is Accepted.

Question 4

Write a C program to check whether a given string belongs to the language defined by the following Context Free Grammar (CFG):

$$S \rightarrow 0S1 \mid \epsilon$$

Language generated: $0^n 1^n$ ($n \geq 0$)

Pseudocode

```
BEGIN
  READ string S ; n <- LENGTH(S) ; i <- 0
  zeros <- 0 ; ones <- 0
  WHILE i < n AND S[i] = '0' DO
    zeros <- zeros + 1 ; i <- i + 1
  WHILE i < n AND S[i] = '1' DO
    ones <- ones + 1 ; i <- i + 1
  IF i != n THEN
    OUTPUT "Not Accepted"; STOP
  IF zeros = ones THEN
    OUTPUT "Accepted"
  ELSE
    OUTPUT "Not Accepted"
END
```

Sample Output

The screenshot shows a C program in a code editor with a 'Run' button. The code counts the number of '0's and '1's in a string and checks if they are equal. The output window shows the input '00001111' and the message 'String is accepted.' followed by 'Code execution Successful'.

```
main.c Run Output
4 int main() {
5     // ...
6     int zeros = 0;
7     int ones = 0;
8
9
10    printf("Enter a string: ");
11    scanf("%s", str);
12
13    n = strlen(str);
14
15    i = 0;
16
17    // Count zeros at beginning
18    while (i < n && str[i] == '0') {
19        zeros++;
20        i++;
21    }
22
23    // ...
24}
```

Enter a string: 00001111
String is accepted.
--- Code execution Successful ---

Explanation

The grammar $S \rightarrow 0S1 \mid \epsilon$ generates equal numbers of 0s followed by an equal number of 1s — the classic non-regular language $0^n 1^n$. The program counts the leading zeros and the following ones and checks that the counts match. For the input 00001111, there are four 0s and four 1s, so it is Accepted.

Question 5

Write a C program to check whether a given string belongs to the language defined by the following Context Free Grammar (CFG):

$$S \rightarrow A101A \quad A \rightarrow 0A \mid 1A \mid \epsilon$$

Language generated: $(0+1)^*101(0+1)^*$

Pseudocode

```
BEGIN
  READ string S ; n <- LENGTH(S)
  FOR i <- 0 TO n-1 DO
    IF S[i] != '0' AND S[i] != '1' THEN
      OUTPUT "Not Accepted"; STOP
  found <- FALSE
  FOR i <- 0 TO n-3 DO
    IF S[i..i+2] = "101" THEN
      found <- TRUE ; BREAK
  IF found THEN
    OUTPUT "Accepted"
  ELSE
    OUTPUT "Not Accepted"
END
```

Sample Output

main.c	Run	Output
<pre> 4 int main() { 22 } 23 24 // Check that only 0 and 1 are present 25 for (i = 0; i < n; i++) { 26 if (str[i] != '0' && str[i] != '1') { 27 found = 0; 28 break; 29 } 30 } 31 32 if (found) 33 printf("String is accepted.\n"); 34 else 35 printf("String is NOT accepted.\n"); 36 37 return 0; 38 } </pre>		<pre> Enter a string: 101101 String is accepted. --- Code Execution Successful --- </pre>

Explanation

The grammar $S \rightarrow A101A$, $A \rightarrow 0A \mid 1A \mid \epsilon$ generates any string that contains the substring 101 somewhere in it, surrounded by arbitrary 0s and 1s — the language $(0+1)^*101(0+1)^*$. The program scans the string for the pattern "101" at any position. For the input 101101, the substring 101 appears at the start, so it is Accepted.

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

88/100

J